PEIT: Personal Economic Inflation Tracker

Decentralizing Economic Measurement Through Verifiable Purchase Analytics

Ken Tanaka Hernández
HAIKU, Guadalajara, Mexico
kentanaka3@outlook.com

September 2026

Abstract

Official Macroeconomic Inflation (MEI) metrics, including the Consumer Price Index (CPI) and Harmonized Indices of Consumer Prices (HICP), aggregate national consumption via centralized statistical weights. While invaluable for monetary policy and macroeconomic surveillance, their aggregate top-down structure imposes a rigid, population-averaged consumption bundle that systematically obscures the idiosyncratic realities of Individual Expenditure Patterns (IEP). Consequently, individuals facing distinct dietary requirements, rural-urban supply discrepancies, or localized merchant pricing encounter a substantial disconnect between official statistical releases and their personal cost of living.

Personal Economic Inflation Tracker (PEIT) resolves this measurement gap through a local-first, privacy-preserving computational architecture for personal price indexing and expenditure tracking. In PEIT, the individual consumer operates as an autonomous, sovereign observation node where sensitive financial transactions remain strictly on-device. The demonstrated system ingests receipt line items structured under an authoritative 24-column schema, validates data types and invariants, maintains an append-only JavaScript Object Notation Lines (JSONL) audit ledger synchronized with an automatic SQLite projection cache, and computes two orthogonal indicators: an unweighted exact-item fixed-basket price index (Carli arithmetic elementary aggregation) and a changing-basket expenditure index.

To provide rigorous evaluation fixtures without compromising personal privacy, PEIT incorporates an advanced synthetic data generator (`peit.profiler`) that cleanly decouples consumer identity signals from market price dynamics. The market pricing engine models latent inflation drift, discrete pack-size volume discount curves, sticky price adjustments via (s, S) menu-cost bands and Calvo review hazard jumps, and dual-mode charm pricing (grid quantization and psychological endings). Consumer purchase cadences are governed by co-living deterministic intervals and wear-and-tear Weibull renewal hazard processes with refractory and ceiling bounds. Transaction streams and index trajectories are inspected via an interactive multi-graph visualizer dashboard (`peit.dashboard`) that performs Bennett exact additive welfare decomposition into price and quantity effects. All system capabilities are verified against an automated test suite comprising 105 unit and integration tests.

Keywords: personal inflation; expenditure tracking; price index; local-first software; sticky pricing; Weibull renewal; Bennett decomposition; receipt analytics

1 Background and motivation

Inflation is fundamentally experienced through the specific goods, quantities, and prices that an economic agent encounters in the marketplace. While national indices summarize purchasing power across millions of households, they cannot answer the personal question: *How has the cost of my idiosyncratic consumption basket evolved over time?* Furthermore, an increase in a consumer's total monthly expenditure may stem from multiple confounding causes: escalating unit shelf prices, expanded consumption volume, brand substitution, or non-recurring one-off purchases. Collapsing these distinct causal mechanisms into a single headline percentage obscures individual financial realities.

Weighting architecture plays a decisive role in this divergence. Let $e_{i,h}$ denote the expenditure of household $h \in \{1, \ldots, H\}$ on item $i \in \{1, \ldots, K\}$. Conventional national indices employ plutocratic weighting, wherein each currency unit exerts equal influence:

$$w_i^{\text{pluto}} = \frac{\sum_{h=1}^{H} e_{i,h}}{\sum_{h=1}^{H} \sum_{k=1}^{K} e_{k,h}}. \tag{1}$$

Under plutocratic weighting, high-expenditure households disproportionately dominate the aggregate basket composition. Conversely, a democratic weighting framework computes budget shares per household before taking the arithmetic mean across the population:

$$w_i^{\text{demo}} = \frac{1}{H} \sum_{h=1}^{H} \frac{e_{i,h}}{\sum_{k=1}^{K} e_{k,h}}. \tag{2}$$

These formulations represent macroeconomic contrasts, not outputs generated by PEIT. They formalize why population-wide indices inevitably depart from individual experience. PEIT replaces national weighting assumptions with a sovereign personal node whose consumption bundle is defined solely by verified, observed transactions.

To preserve analytical integrity, PEIT enforces a strict separation between the *offer side* (market pricing conditions faced by the consumer) and the *demand side* (observed purchasing decisions and volumes).

Table 1: Measurement dichotomy: Macroeconomic indices versus PEIT personal node.

Dimension	Macroeconomic Index (CPI / HICP)	PEIT Prototype Perspective
Weighting	Plutocratic national expenditure shares (w_i^{pluto})	Observed single-node transaction stream
Basket Definition	Fixed cohort or representative consumer bundle	Exact normalized items observed in base month (t_0)
Data Capture	Scanner feeds, retailer surveys, web scraping	Local receipt rows (manual or synthetic)
Privacy Domain	Centralized government / institutional database	Strictly local-first on-device execution
Analytical Target	Economy-wide purchasing power and monetary anchor	Orthogonal personal price and expenditure indices

Scope & Boundaries: Contrast between macro-level statistical reporting and PEIT's single-node micro-accounting architecture. PEIT produces derived personal indicators, not official statistics.

Table 2: Decoupled offer and demand measurement dimensions in PEIT.

Measurement	Principal Evaluated Question	Underlying Physical Drivers
Price / Offer	What does a constant basket of goods cost?	Shelf unit prices, statutory taxes, merchant discounts
Expenditure / Demand	How much currency did the agent disburse?	Unit prices, purchase volume, pack sizes, one-offs

Analytical Clarity: Isolating price effects from volume effects prevents behavioral budget adjustments from being misinterpreted as price stability.

2 Scientific and technological foundations

PEIT models each receipt line as an independent empirical observation. All arithmetic follows a signed double-entry ledger convention: consumption expenditures are recorded as negative paid totals ($a_j < 0$), gross incomes and refunds are positive ($a_j > 0$), promotional discounts are positive offsets ($+$), while fees, taxes, and deposit surcharges are negative adjustments ($-$).

Notation and Mathematical Formulation. Let $\mathcal{B}_t$ denote the set of normalized items purchased in month t, and let $\mathcal{K}_{i,t}$ denote the set of purchase transactions for item $i \in \mathcal{B}_t$ during month t. For transaction $k \in \mathcal{K}_{i,t}$, let $q_{i,t,k} \in \mathbb{R}^+$ represent the physical volume (quantity $\times$ pack size), and let $p_{i,t,k} \in \mathbb{R}^+$ be the observed unit price. The quantity-weighted average unit price for item i in month t is:

$$\bar{p}_{i,t} = \frac{\sum_{k \in \mathcal{K}_{i,t}} q_{i,t,k}\, p_{i,t,k}}{\sum_{k \in \mathcal{K}_{i,t}} q_{i,t,k}}. \tag{3}$$

Let $t = 0$ denote the baseline observation month. The set of matched items between base month 0 and current month t is $\mathcal{M}_t = \{i \in \mathcal{B}_0 \cap \mathcal{B}_t \mid \bar{p}_{i,0} > 0\}$. For each matched item, the price relative is $r_{i,t} = \bar{p}_{i,t}/\bar{p}_{i,0}$. The elementary price aggregation implemented in `src/peit/core.py` is the unweighted arithmetic mean of matched price relatives (the Carli index):

$$I_t^{\text{price}} = 100 \times \frac{1}{|\mathcal{M}_t|} \sum_{i \in \mathcal{M}_t} \frac{\bar{p}_{i,t}}{\bar{p}_{i,0}}. \tag{4}$$

To evaluate match completeness, PEIT computes the basket coverage metric:

$$C_t = 100 \times \frac{|\mathcal{M}_t|}{|\mathcal{B}_0|}. \tag{5}$$

Orthogonally, the changing-basket paid-expenditure index measures nominal cash outflows over all consumption transactions $\mathcal{T}_t = \{j \in t \mid \text{event_type}_j \in \{\texttt{recurring}, \texttt{one_off}\}\}$:

$$I_t^{\text{exp}} = 100 \times \frac{\sum_{j \in \mathcal{T}_t} |a_j|}{\sum_{j \in \mathcal{T}_0} |a_j|}. \tag{6}$$

Sequential month-over-month rate changes are evaluated as $\text{MoM}_t = 100 \times (I_t^{\text{price}}/I_{t-1}^{\text{price}} - 1)$.

Bennett Exact Additive Welfare Decomposition. When analyzing expenditure shifts between consecutive months $t-1$ and t, `src/peit/dashboard.py` evaluates the Bennett additive decomposition across matched items $\mathcal{M}_t$:

$$\Delta E_t = \sum_{i \in \mathcal{M}_t} \bar{p}_{i,t} q_{i,t} - \sum_{i \in \mathcal{M}_t} \bar{p}_{i,t-1} q_{i,t-1} = \Delta P_t + \Delta Q_t, \tag{7}$$

where the price effect ΔP_t and quantity effect ΔQ_t are defined by arithmetic-average weights:

$$\Delta P_t = \sum_{i \in \mathcal{M}_t} \left(\frac{q_{i,t-1} + q_{i,t}}{2} \right) (\bar{p}_{i,t} - \bar{p}_{i,t-1}), \qquad \Delta Q_t = \sum_{i \in \mathcal{M}_t} \left(\frac{\bar{p}_{i,t-1} + \bar{p}_{i,t}}{2} \right) (q_{i,t} - q_{i,t-1}). \tag{8}$$

The identity $\Delta P_t + \Delta Q_t \equiv \Delta E_t$ holds unconditionally with exact zero residual.

Market Dynamics and Sticky Price Formation. In the synthetic profiler engine (`src/peit/profiler/market.py`), market prices reflect realistic macroeconomic stickiness rather than unconstrained continuous random walks. For catalog item i at merchant m in month t, the unobserved latent target price $P^*_{i,m,t}$ incorporates compound annual inflation drift π_i and Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma_i^2)$:

$$P^*_{i,m,t} = P_{0,i} \cdot (1+\pi_i)^{t/12} \cdot \exp(\varepsilon) \cdot \mu(s) \cdot \gamma_m(t), \tag{9}$$

where $\mu(s)$ is a discrete volume discount multiplier for pack size $s \in \mathbb{N}^+$, and $\gamma_m(t)$ is an optional seasonal factor. To model menu costs and staggered nominal price reviews, PEIT implements an (s, S) threshold band with a Calvo hazard jump. Let $p^{\text{shelf}}_{i,m}$ denote the current shelf price tracked per (i, m) store-item pair. A price review is triggered if a Calvo random shock succeeds with probability $\lambda_{\text{calvo}} \in [0, 1]$ or if the relative deviation exceeds the menu-cost band $\theta_{\text{menu}} > 0$:

$$\text{Review Triggered} \iff (U_{\text{calvo}} < \lambda_{\text{calvo}}) \vee \left(\frac{|P^*_{i,m,t} - p^{\text{shelf}}_{i,m}|}{p^{\text{shelf}}_{i,m}} \geq \theta_{\text{menu}} \right). \tag{10}$$

If neither condition is satisfied, the shelf price remains locked: $p_{i,m,t} = p^{\text{shelf}}_{i,m}$.

Upon review, target prices undergo dual-mode charm pricing:

1. **Grid Quantization:** The price snaps to a defined currency increment $\delta_{\text{grid}} > 0$ (e.g., 0.50): $p_{\text{quant}} = \delta_{\text{grid}} \cdot \text{round}(P^*/\delta_{\text{grid}})$.
2. **Psychological Charm Snapping:** The price snaps to the nearest configured psychological cent ending $c \in \mathcal{C}$ (e.g., $\{0.49, 0.95, 0.99\}$):

$$p_{\text{final}} = \arg\min_{y \in \mathcal{Y}} |y - p_{\text{quant}}|, \quad \mathcal{Y} = \left\{ d + \frac{c}{100} \;\middle|\; d \in \{\lfloor p_{\text{quant}} \rfloor - 1, \lfloor p_{\text{quant}} \rfloor, \lfloor p_{\text{quant}} \rfloor + 1\}, c \in \mathcal{C}, y > 0 \right\}. \tag{11}$$

Consumer Renewal and Weibull Hazard Scheduling. Consumer purchase cadences in `peit.profiler` co-live across three modes: fixed monthly count, periodic deterministic intervals, and wear-and-tear renewal hazard processes. In hazard mode, let $\Delta t = t - t_{\text{last}}$ be the months elapsed since the last purchase. The renewal hazard $h(\Delta t)$ is bounded by an absolute refractory period $\tau_{\min}$ and a mandatory renewal ceiling $\tau_{\max}$:

$$h(\Delta t) = \begin{cases} 0, & \text{if } \Delta t < \tau_{\min}, \\ 1, & \text{if } \Delta t \geq \tau_{\max}, \\ \frac{F(\Delta t) - F(\Delta t - 1)}{1 - F(\Delta t - 1)}, & \text{if } \tau_{\min} \leq \Delta t < \tau_{\max}, \end{cases} \tag{12}$$

where $F(\tau) = 1 - \exp(-(\tau/\eta)^k)$ is the cumulative Weibull distribution with shape parameter $k = \kappa_{\text{hazard}}$ ($k > 1$ models increasing wear-and-tear failure) and scale parameter $\eta = \bar{T}/\Gamma(1 + 1/k)$, with $\bar{T}$ representing the expected renewal interval.

Table 3: Elementary and upper-level price index formulas considered by PEIT.

Estimator	Mathematical Definition	Time Rev.[a]	Weights	Implementation Status
Carli Elementary	$P^{\text{Carli}}_{0,t} = \frac{1}{n}\sum_{i=1}^{n} \frac{p_{i,t}}{p_{i,0}}$	Fails	Unweighted	Implemented Baseline (`src/peit/core.py`)
Jevons Elementary	$P^{\text{Jevons}}_{0,t} = \prod_{i=1}^{n} \left(\frac{p_{i,t}}{p_{i,0}}\right)^{1/n}$	Holds	Unweighted	Candidate Alternative (Design review)
Laspeyres Upper	$P^{\text{L}}_{0,t} = \frac{\sum p_{i,t} q_{i,0}}{\sum p_{i,0} q_{i,0}}$	Fails	Base period	Planned Roadmap Milestone
Fisher Superlative	$P^{\text{F}}_{0,t} = \sqrt{P^{\text{L}}_{0,t} \cdot P^{\text{P}}_{0,t}}$	Holds	Symmetric	Planned Benchmark Standard

[a] The time reversal test requires $P_{0,t} \cdot P_{t,0} = 1$. The Carli index exhibits systematic upward bounce under oscillating price vectors; its current selection prioritizes auditing simplicity.

3 System architecture and data provenance

PEIT is organized into modular Python components backed by verifiable local storage. Figure 1 illustrates the complete end-to-end data pipeline according to formal visual evidence boundaries.

Dual-Storage Persistence and Self-Healing Cache. Storage in PEIT (`src/peit/ledger.py`) combines an authoritative append-only JSONL text log with an automatic sibling SQLite projection cache (`ledger.db`):

1. **Authoritative Append-Only JSONL:** Every transaction entry is serialized as an immutable, single-line JSON record. This guarantees human inspectability, simple disaster recovery, and zero write amplification.
2. **Sibling SQLite Projection Cache:** A normalized relational table index speeds multi-attribute queries (by month, category, merchant, account, or event type), running balance calculations, and category breakdowns.
3. **Self-Healing Cache Invalidation:** The database tracks the JSONL file's modification timestamp (`st_mtime`) and byte size (`st_size`) in an internal `meta` table. If manual external edits, file truncation, or database deletion occur, the cache automatically self-heals by dropping and reconstructing the entire SQLite projection from the authoritative JSONL log.
4. **Deterministic Deduplication:** Idempotency across repeated imports is enforced via a deterministic SHA-256 digest:

$$\text{entry_id} = \text{SHA256}\Big(\texttt{receipt_id} \parallel \texttt{item} \parallel \text{round}(\texttt{paid_total}, 6) \parallel \texttt{receipt_date}\Big). \tag{13}$$

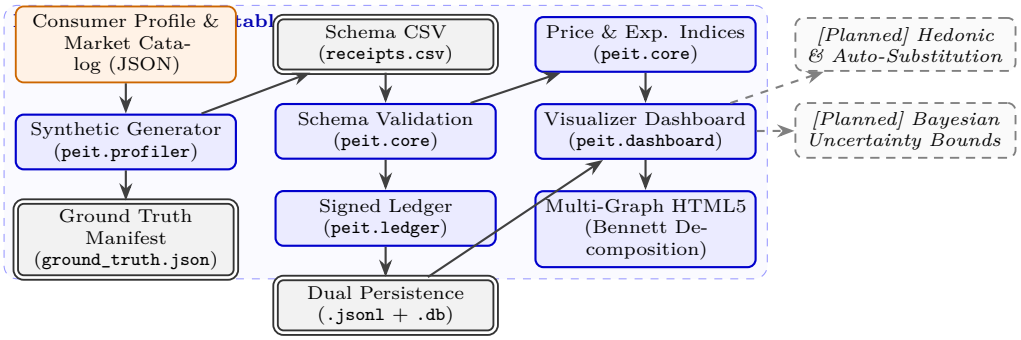


Figure 1: PEIT system architecture and end-to-end data pipeline. What the reader should notice: Implemented Python modules (solid blue borders) execute locally; immutable storage artifacts (double borders) maintain synchronized JSONL logs and SQLite projection caches; speculative analytical extensions (dashed gray borders) are explicitly partitioned with [Planned] badges. Evidence anchor: Automated test fixtures in `test/`.

4 Data modeling and schema specification

Table ?? defines the complete 24-column receipt data dictionary, specifying strict data types, regular expression constraints, sign conventions, and epistemic classifications.

Table 4: Receipt 24-column data dictionary, validation constraints, and operational bindings.

Field Name	Data Type	Domain / Format	Sign[a]	Epistemic Class & Operational Semantics
`receipt_id`	String	Regex `^[A-Za-z0-9_\-]+$`	—	[Observed (Synthetic)] Unique purchase occasion transaction identifier
`receipt_date`	ISO Date	`YYYY-MM-DD`	—	[Observed (Synthetic)] Observation date within target calendar month
`merchant`	String	Non-empty string	—	[Observed (Synthetic)] Vendor or retailer legal / trade name
`item`	String	Lowercase canonical	—	[Analyst-Normalized] Exact matching key
`brand`	String	Non-empty string / blank	—	[Observed (Synthetic)] Product brand or manufacturer name (Schema)
`category`	String	COICOP taxonomy class	—	[Analyst-Normalized] Macro spending classification (e.g., `Food and beverages`)
`quantity`	Numeric	Integer / Float > 0	—	[Observed (Synthetic)] Number of discrete packages purchased ($q_{i,t,k}$)
`pack_size`	Numeric	Integer / Float > 0	—	[Observed (Synthetic)] Number of physical units per package ($s_{i,t,k}$)
`unit`	String	Non-empty string	—	[Analyst-Normalized] Physical unit of measurement (`L`, `kg`, `piece`)
`unit_price`	Numeric	Float ≥ 0.0	+	[Observed (Synthetic)] Unit shelf price per physical measure ($p_{i,t,k}$)
`currency`	String	Regex `^[A-Za-z]{3}$`	—	[Observed (Synthetic)] ISO-4217 three-letter currency code (e.g., `MXN`)
`discount`	Numeric	Float ≥ 0.0	+	[Observed (Synthetic)] Promotional discount offset (money saved)
`fee`	Numeric	Float ≤ 0.0	−	[Observed (Synthetic)] Service fee, markup, or delivery surcharge
`tax`	Numeric	Float ≤ 0.0	−	[Observed (Synthetic)] Value-added or indirect transaction tax
`deposit`	Numeric	Float	±	[Observed (Synthetic)] Container deposit (liability − or redemption +)
`paid_total`	Numeric	Float	±	[Derived / Accounting] Net transaction cash flow (− expenditure, + income)
`event_type`	Enum	Allowed status set[b]	—	[Analyst-Normalized] Transaction classification for index filtering
`store_tier`	Enum	Allowed tier set[c]	—	[Analyst-Normalized] Store / vendor quality bracket (`A`, `B`, `C`, `D`, `E`)
`store_rating`	Numeric	Float $\in [1.0, 5.0]$	—	[Observed (Synthetic)] Store service / satisfaction evaluation score (1.0–5.0)
`brand_tier`	Enum	Allowed tier set[c]	—	[Analyst-Normalized] Brand / manufacturer reputation bracket (`A`, `B`, `C`, `D`, `E`)
`brand_rating`	Numeric	Float $\in [1.0, 5.0]$	—	[Observed (Synthetic)] Brand evaluation / satisfaction score (1.0–5.0)
`product_tier`	Enum	Allowed tier set[c]	—	[Analyst-Normalized] Specific product grade / quality bracket (`A`, `B`, `C`, `D`, `E`)
`product_rating`	Numeric	Float $\in [1.0, 5.0]$	—	[Observed (Synthetic)] Consumer satisfaction score for product (1.0–5.0)
`notes`	String	Free-text UTF-8	—	[Observed (Synthetic)] Contextual audit annotations and provenance

[a] Sign convention: (+) denotes incoming revenue, discount savings, or asset expansion; (−) denotes cash outflow, fee, tax surcharge, or expenditure liability.
[b] Allowed `event_type` values: `recurring`, `one_off` (consumption events included in index calculations), `income`, `transfer`, `refund` (financial events tracked in ledger balances but excluded from price index relatives).
[c] Allowed tier values (`store_tier`, `brand_tier`, `product_tier`): `A`, `B`, `C`, `D`, `E`. Satisfaction ratings (`store_rating`, `brand_rating`, `product_rating`) are floating-point scores bounded strictly in $[1.0, 5.0]$ with 1-decimal precision.

5 Demonstration scenarios and worked numerical examples

Schema Worked Numerical Example. Consider a synthetic household purchasing whole milk in Guadalajara, Mexico across two consecutive observation months (t_0 and t_1):

- **Base Month (t_0):** A single pack of whole milk is purchased: `receipt_id` = `"REC-202601-00001"`, `receipt_date` = `"2026-01-15"`, `merchant` = `"SuperMarket Alpha"`, `item` = `"milk 1 l"`, `brand` = `"BrandA"`, `category` = `"Food and beverages"`, `quantity` = 1, `pack_size` = 1, `unit` = `"L"`, `unit_price` = 2.50, `currency` = `"MXN"`, `discount` = 0.00, `paid_total` = −2.50, `event_type` = `"recurring"`, `store_tier` = `"A"`, `store_rating` = 5.0, `brand_tier` = `"A"`, `brand_rating` = 5.0, `product_tier` = `"A"`, `product_rating` = 5.0. Thus, base unit price is $\bar{p}_{\text{milk},0} = 2.50\,\text{MXN}$.
- **Current Month (t_1), Scenario A (Pure Price Inflation):** Unit shelf price rises to MXN 3.00, and the consumer buys 1 pack (paid_total = −3.00 MXN). The price relative is $r = 3.00/2.50 = 1.20$. The elementary price index rises to $I_1^{\text{price}} = 100 \times 1.20 = 120.0$. Because volume is unchanged, the expenditure index is also $I_1^{\text{exp}} = 100 \times (3.00/2.50) = 120.0$.
- **Current Month (t_1), Scenario B (Price Inflation + Volume Surge):** Unit shelf price rises to MXN 3.00, but the consumer purchases 2 packs (quantity = 2, paid_total = −6.00 MXN). Unit shelf price remains 3.00 MXN, maintaining $I_1^{\text{price}} = 120.0$. However, total expenditure surges to 6.00 MXN, resulting in $I_1^{\text{exp}} = 100 \times (6.00/2.50) =$

Table 5: System capability verification and evidence provenance matrix.

System Feature	Status	Executable Source	Empirical Evidence / Test Anchor
Synthetic Generation	Implemented	`src/peit/profiler/`	`test/test_profiler.py` (39 tests: (s, S) menu costs, Calvo jumps, Weibull renewal, charm pricing)
Schema Ingestion	Implemented	`src/peit/core.py`	`test/test_core.py` (7 tests: 24-column validation, pack size rules, date formats, weighted unit price)
Dual-Storage Persistence	Implemented	`src/peit/ledger.py`	`test/test_ledger.py` (24 tests: SHA-256 deduplication, atomic dual-write, mtime cache self-healing)
Elementary Price Indices	Implemented	`src/peit/core.py`	`test/test_core.py` (Exact Carli price index, changing-basket expenditure index, coverage metric)
Multi-Graph Visualizer	Implemented	`src/peit/dashboard.py`	`test/test_dashboard.py` (14 tests: Bennett exact welfare decomposition, category PPC/ADI, HTML5 export)
CLI Orchestration	Implemented	`src/peit/cli.py`	`test/test_cli.py` (13 tests: end-to-end import, query, breakdown, and dashboard generation)
Automated Item Matching	Planned	Conceptual design	Future roadmap milestone for non-exact product substitution
Uncertainty Quantification	Planned	Literature baseline	Future roadmap milestone for bootstrap and Bayesian credible intervals

Verification Standard: Implemented capabilities are verified against automated unit tests executed via `pytest test/ -v` (105 passing assertions). Planned features indicate design objectives.

240.0.

- **Bennett Exact Welfare Decomposition on Scenario B:** Between t_0 and t_1, total nominal expenditure increases by $\Delta E = 6.00 - 2.50 = +3.50\,\text{MXN}$. Applying the Bennett decomposition:

$$\begin{aligned} \Delta P &= \left(\frac{q_0 + q_1}{2}\right)(p_1 - p_0) = \left(\frac{1+2}{2}\right)(3.00 - 2.50) = 1.50 \times 0.50 = +0.75\,\text{MXN}, \\ \Delta Q &= \left(\frac{p_0 + p_1}{2}\right)(q_1 - q_0) = \left(\frac{2.50 + 3.00}{2}\right)(2 - 1) = 2.75 \times 1.00 = +2.75\,\text{MXN}. \end{aligned} \tag{14}$$

  Checking the exact additive invariant: $\Delta P + \Delta Q = 0.75 + 2.75 = 3.50 = \Delta E$, yielding an exact zero residual ($\varepsilon = 0.00$). This proves that $+0.75\,\text{MXN}$ of the monthly increase was driven by pure price inflation, whereas $+2.75\,\text{MXN}$ reflected volume expansion.

Reproducible Command-Line Workflow. In live demonstrations, visitors can execute the end-to-end pipeline:

```
# 1. Generate 12-month synthetic receipt stream under Schema with seed reproducibility:
python3 -m peit.profiler generate --profile config/profiles/alice.json \
  --market config/markets/mexico_2024.json --seed 42 -o data/Alice/receipts.csv

# 2. Ingest and validate Schema receipts into the authoritative JSONL and SQLite ledger:
PYTHONPATH=src python3 -m peit.cli --ledger data/Alice/ledger.jsonl --import data/Alice/receipts.csv

# 3. Query high-level financial balances and spending distributions by category:
PYTHONPATH=src python3 -m peit.cli --ledger data/Alice/ledger.jsonl --balance
PYTHONPATH=src python3 -m peit.cli --ledger data/Alice/ledger.jsonl --breakdown

# 4. Generate multi-graph interactive HTML5 dashboard with Bennett decomposition:
PYTHONPATH=src python3 -m peit.cli --ledger data/Alice/ledger.jsonl --html doc/showcase/dashboard.html
```

6 Novelty, epistemic boundaries, and limitations

PEIT's primary contribution is methodological transparency: it disaggregates personal inflation into observable market conditions, consumer behavioral choices, and nominal cash flows. The local-first design guarantees that sensitive personal consumption records never leave the user's computing environment.

Epistemic Boundaries and Verification Standards. To avoid misrepresenting computational heuristics as established macroeconomic facts, PEIT enforces strict epistemic boundaries:

- **Raw Observations vs. Analyst Normalizations:** Canonical matching keys (`item`) and classifications (`category`) are labeled as analyst normalizations.
- **Axiomatic Properties of the Carli Index:** The implemented elementary price index uses the unweighted Carli formula. While auditable and simple, the Carli index violates the time reversal axiom ($P^{\text{Carli}}_{0,t} \cdot P^{\text{Carli}}_{t,0} \geq 1$) and is susceptible to upward bounce under fluctuating prices. Transitioning to a geometric Jevons elementary index or a superlative Fisher index is a planned methodological enhancement.
- **Exact-Match Constraint:** PEIT currently requires exact matches on normalized item keys. Unmatched items are excluded from I^{price}_t and documented via the coverage metric C_t. Automatic hedonic quality adjustments, product substitution models, and machine-learning entity resolution remain future roadmap targets.

7 Conclusion and future roadmap

PEIT provides a rigorous, verifiable foundation for personal economic inflation tracking. By maintaining an immutable JSONL audit trail, synchronized SQLite projections, explicit 24-column schema validation, and decoupled price and expenditure indices, the system demonstrates that individual cost-of-living measurement can be achieved without compromising personal privacy. Future extensions will incorporate automated hedonic regression, category-level fallback matching, Bayesian credible intervals for price relatives, and multi-currency exchange adjustments.